TP7 - Formulaires de Recherche - Symfony 7

Prérequis

Base de données configurée avec les entités Article et Category (TP4-TP6)

Objectifs du TP

1. Créer un formulaire de recherche par nom sur la page d'accueil
2. Créer un formulaire de recherche par catégorie
3. Créer un formulaire de recherche par prix (fourchette)
4. Ajouter les liens de recherche dans la navigation

Étape 1: Création des entités de recherche

Les entités de recherche sont des classes simples qui ne sont pas mappées à la base de données (pas d'annotations ORM).

1.1 Entité PropertySearch (Recherche par nom)

Fichier: `src/Entity/PropertySearch.php`

```php
<?php

namespace App\Entity;

use Symfony\Component\Validator\Constraints as Assert;

class PropertySearch
{
    private ?string $nom = null;

    public function getNom(): ?string
    {
        return $this->nom;
    }

    public function setNom(?string $nom): self
    {
        $this->nom = $nom;
        return $this;
    }
}
```

1.2 Entité CategorySearch (Recherche par catégorie)

Fichier: `src/Entity/CategorySearch.php`

```php
<?php

namespace App\Entity;

use App\Entity\Category;

class CategorySearch
```

```php
{
    private ?Category $category = null;

    public function getCategory(): ?Category
    {
        return $this->category;
    }

    public function setCategory(?Category $category): self
    {
        $this->category = $category;
        return $this;
    }
}
```

1.3 Entité PriceSearch (Recherche par prix)

Fichier: `src/Entity/PriceSearch.php`

```php
<?php

namespace App\Entity;

class PriceSearch
{
    private ?int $minPrice = null;
    private ?int $maxPrice = null;

    public function getMinPrice(): ?int
    {
        return $this->minPrice;
    }

    public function setMinPrice(?int $minPrice): self
    {
        $this->minPrice = $minPrice;
        return $this;
    }

    public function getMaxPrice(): ?int
    {
        return $this->maxPrice;
    }

    public function setMaxPrice(?int $maxPrice): self
    {
        $this->maxPrice = $maxPrice;
        return $this;
    }
}
```

Étape 2: Création des types de formulaire

2.1 PropertySearchType (Recherche par nom)

Fichier: `src/Form/PropertySearchType.php`

```php
<?php

namespace App\Form;

use App\Entity\PropertySearch;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
use Symfony\Component\Form\Extension\Core\Type\TextType;

class PropertySearchType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('nom', TextType::class, [
                'label' => false,
                'attr' => [
                    'class' => 'form-control',
                    'placeholder' => 'Rechercher par nom...'
                ],
                'required' => false
            ]);
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => PropertySearch::class,
        ]);
    }
}
```

2.2 CategorySearchType (Recherche par catégorie)

Fichier: `src/Form/CategorySearchType.php`

```php
<?php

namespace App\Form;

use App\Entity\CategorySearch;
use App\Entity\Category;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
use Symfony\Bridge\Doctrine\Form\Type\EntityType;

class CategorySearchType extends AbstractType
{
```

```
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('category', EntityType::class, [
                'class' => Category::class,
                'choice_label' => 'titre',
                'label' => 'Catégorie',
                'attr' => ['class' => 'form-control'],
                'required' => false,
                'placeholder' => 'Toutes les catégories'
            ]);
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => CategorySearch::class,
        ]);
    }
}
```

2.3 PriceSearchType (Recherche par prix)

Fichier: `src/Form/PriceSearchType.php`

```php
<?php

namespace App\Form;

use App\Entity\PriceSearch;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
use Symfony\Component\Form\Extension\Core\Type\IntegerType;

class PriceSearchType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('minPrice', IntegerType::class, [
                'label' => 'Prix min',
                'attr' => [
                    'class' => 'form-control',
                    'placeholder' => 'Prix minimum'
                ],
                'required' => false
            ])
            ->add('maxPrice', IntegerType::class, [
                'label' => 'Prix max',
                'attr' => [
                    'class' => 'form-control',
                    'placeholder' => 'Prix maximum'
                ],
```

```
                'required' => false
            ]);
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => PriceSearch::class,
        ]);
    }
}
```

Étape 3: Ajout des routes de recherche

Modifiez le contrôleur `IndexController` pour ajouter les nouvelles routes.
Fichier: `src/Controller/IndexController.php`
Ajoutez les imports nécessaires:

```
use App\Form\PropertySearchType;
use App\Form\CategorySearchType;
use App\Form\PriceSearchType;
```

3.1 Recherche par nom sur la page d'accueil

```
#[Route('/', name: 'article_list')]
public function home(Request $request): Response
{
    $propertySearch = new PropertySearch();
    $form = $this->createForm(PropertySearchType::class, $propertySearch);
    $form->handleRequest($request);

    $articles = [];
    if ($form->isSubmitted() && $form->isValid()) {
        $nom = $propertySearch->getNom();
        $articles =
$this->entityManager->getRepository(Article::class)->findBy(['nom' => $nom]);
    } else {
        $articles = $this->entityManager->getRepository(Article::class)->findAll();
    }

    return $this->render('articles/index.html.twig', [
        'articles' => $articles,
        'form' => $form->createView()
    ]);
}
```

3.2 Route de recherche par catégorie

```
#[Route('/article/search/category', name: 'article_search_category')]
public function searchByCategory(Request $request): Response
{
    $categorySearch = new CategorySearch();
    $form = $this->createForm(CategorySearchType::class, $categorySearch);
    $form->handleRequest($request);
```

```
    $articles = [];
    if ($form->isSubmitted() && $form->isValid()) {
        $category = $categorySearch->getCategory();
        if ($category) {
            $articles =
$this->entityManager->getRepository(Article::class)->findBy(['category' =>
$category]);
        }
    }

    return $this->render('articles/searchCategory.html.twig', [
        'articles' => $articles,
        'form' => $form->createView()
    ]);
}
```

3.3 Route de recherche par prix

```
#[Route('/article/search/price', name: 'article_search_price')]
public function searchByPrice(Request $request): Response
{
    $priceSearch = new PriceSearch();
    $form = $this->createForm(PriceSearchType::class, $priceSearch);
    $form->handleRequest($request);

    $articles = [];
    if ($form->isSubmitted() && $form->isValid()) {
        $minPrice = $priceSearch->getMinPrice();
        $maxPrice = $priceSearch->getMaxPrice();

        $articles = $this->entityManager->getRepository(Article::class)->findAll();
        $filteredArticles = [];

        foreach ($articles as $article) {
            $prix = $article->getPrix();
            $passesMin = $minPrice === null || $prix >= $minPrice;
            $passesMax = $maxPrice === null || $prix <= $maxPrice;
            if ($passesMin && $passesMax) {
                $filteredArticles[] = $article;
            }
        }
        $articles = $filteredArticles;
    }

    return $this->render('articles/searchPrice.html.twig', [
        'articles' => $articles,
        'form' => $form->createView()
    ]);
}
```

Étape 4: Mise à jour des templates

4.1 Template index.html.twig (Recherche par nom)

Ajoutez le formulaire de recherche en haut de la page:

```
{% extends 'base.html.twig' %}
{% block title %}Liste des Articles{% endblock %}

{% block body %}
<div class="container mt-4">
    <h1>Liste des Articles</h1>

    {# Search by name form #}
    <div class="row mb-4">
        <div class="col-md-6">
            {{ form_start(form) }}
            <div class="input-group">
                {{ form_widget(form.nom, {'attr': {'class': 'form-control',
'placeholder': 'Rechercher par nom...'}}) }}
                <button type="submit" class="btn btn-primary">Rechercher</button>
            </div>
            {{ form_end(form) }}
        </div>
    </div>

    {# Table des articles... #}
</div>
{% endblock %}
```

4.2 Template searchCategory.html.twig

Fichier: `templates/articles/searchCategory.html.twig`

```
{% extends 'base.html.twig' %}

{% block title %}Recherche par catégorie{% endblock %}

{% block body %}
<div class="container mt-4">
    <h1>Recherche par catégorie</h1>

    <div class="row mb-4">
        <div class="col-md-6">
            {{ form_start(form) }}
            {{ form_widget(form) }}
            <button type="submit" class="btn btn-primary mt-2">Rechercher</button>
            {{ form_end(form) }}
        </div>
    </div>

    {% if articles is not empty %}
    <h2 class="mt-4">Résultats ({{ articles|length }})</h2>
    <div class="row">
        {% for article in articles %}
        <div class="col-md-4 mb-4">
```

```
            <div class="card">
                <div class="card-body">
                    <h5 class="card-title">{{ article.nom }}</h5>
                    <p class="card-text">Prix: {{ article.prix }} €</p>
                    {% if article.category %}
                    <p class="card-text"><small class="text-muted">Catégorie: {{
article.category.titre }}</small></p>
                    {% endif %}
                    <a href="{{ path('article_show', {id: article.id}) }}"
class="btn btn-sm btn-info">Voir</a>
                </div>
            </div>
        </div>
        {% endfor %}
    </div>
    {% elseif form.vars.submitted %}
    <div class="alert alert-warning">Aucun article trouvé pour cette
catégorie.</div>
    {% endif %}

    <a href="{{ path('article_list') }}" class="btn btn-secondary mt-3">Retour à la
liste</a>
  </div>
  {% endblock %}
```

4.3 Template searchPrice.html.twig

Fichier: `templates/articles/searchPrice.html.twig`

```
{% extends 'base.html.twig' %}

{% block title %}Recherche par prix{% endblock %}

{% block body %}
<div class="container mt-4">
    <h1>Recherche par prix</h1>

    <div class="row mb-4">
        <div class="col-md-6">
            {{ form_start(form) }}
            <div class="mb-3">
                {{ form_label(form.minPrice) }}
                {{ form_widget(form.minPrice, {'attr': {'class': 'form-control',
'placeholder': 'Prix minimum'}}) }}
            </div>
            <div class="mb-3">
                {{ form_label(form.maxPrice) }}
                {{ form_widget(form.maxPrice, {'attr': {'class': 'form-control',
'placeholder': 'Prix maximum'}}) }}
            </div>
            <button type="submit" class="btn btn-primary">Rechercher</button>
            {{ form_end(form) }}
        </div>
    </div>
```

```
    {% if articles is not empty %}
    <h2 class="mt-4">Résultats ({{ articles|length }})</h2>
    <div class="row">
        {% for article in articles %}
        <div class="col-md-4 mb-4">
            <div class="card">
                <div class="card-body">
                    <h5 class="card-title">{{ article.nom }}</h5>
                    <p class="card-text">Prix: {{ article.prix }} €</p>
                    {% if article.category %}
                    <p class="card-text"><small class="text-muted">Catégorie: {{
article.category.titre }}</small></p>
                    {% endif %}
                    <a href="{{ path('article_show', {id: article.id}) }}"
class="btn btn-sm btn-info">Voir</a>
                </div>
            </div>
        </div>
        {% endfor %}
    </div>
    {% elseif form.vars.submitted %}
    <div class="alert alert-warning">Aucun article trouvé dans cette fourchette de
prix.</div>
    {% endif %}

    <a href="{{ path('article_list') }}" class="btn btn-secondary mt-3">Retour à la
liste</a>
  </div>
  {% endblock %}
```

Étape 5: Mise à jour de la navigation

Modifiez le fichier `templates/inc/navbar.html.twig` pour ajouter le menu déroulant de recherche:

```
  <nav class="navbar navbar-expand-lg navbar-light bg-light">
    <div class="container">
        <a href="/" class="navbar-brand">My Name</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#mobile-nav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="mobile-nav">
            <ul class="navbar-nav me-auto">
                <li class="nav-item">
                    <a href="{{ path('article_list') }}"
class="nav-link">Articles</a>
                </li>
                <li class="nav-item">
                    <a href="{{ path('new_article') }}" class="nav-link">Ajouter
article</a>
                </li>
                <li class="nav-item">
```

```
                    <a href="{{ path('new_category') }}" class="nav-link">Ajouter
catégorie</a>
                </li>
                <li class="nav-item dropdown">
                    <a class="nav-link dropdown-toggle" href="#"
id="navbarDropdown" role="button" data-bs-toggle="dropdown" aria-expanded="false">
                        Rechercher
                    </a>
                    <ul class="dropdown-menu" aria-labelledby="navbarDropdown">
                        <li><a class="dropdown-item" href="{{
path('article_search_category') }}">Par catégorie</a></li>
                        <li><a class="dropdown-item" href="{{
path('article_search_price') }}">Par prix</a></li>
                    </ul>
                </li>
            </ul>
        </div>
    </div>
</nav>
```

Étape 6: Validation et test

Configuration sans Docker (Méthode locale)

Serveur local (alternatives)

```
# Option 1: PHP built-in server (simple)
php -S 127.0.0.1:8000 -t public

# Option 2: Symfony CLI (recommandé)
symfony server:start

# Arrêter le serveur Symfony
symfony server:stop
```

Base de données locale

```
# Créer la base de données
php bin/console doctrine:database:create

# Mettre à jour le schéma
php bin/console doctrine:schema:update --force

# Créer et appliquer les migrations
php bin/console make:migration
php bin/console doctrine:migrations:migrate

# Vérifier le schéma
php bin/console doctrine:schema:info
```

6.1 Vérification de la syntaxe PHP

```
# Docker
```

```
docker compose exec php-fpm php -l
/var/www/symfony-app/src/Controller/IndexController.php

# Local
php -l src/Controller/IndexController.php
```

6.2 Test des routes

```
# Page d'accueil avec formulaire de recherche
curl http://localhost/

# Recherche par catégorie
curl http://localhost/article/search/category

# Recherche par prix
curl http://localhost/article/search/price

# Test local
php -S 127.0.0.1:8000 -t public
# Puis curl http://127.0.0.1:8000/
```

6.3 Vérification du cache

```
# Docker
docker compose exec php-fpm php /var/www/symfony-app/bin/console cache:clear

# Local
php bin/console cache:clear

# Production
APP_ENV=prod APP_DEBUG=0 php bin/console cache:clear
```

Commandes utiles

```
# Lister les routes
php bin/console debug:router

# Vérifier les entités
php bin/console doctrine:schema:info

# Requête SQL directe
php bin/console doctrine:query:sql "SELECT * FROM article"
```